

Un rato para pensarlo y después lo charlamos en el P.L. de la mañana
Rendir solo pero creo que sirve para practicar Suerte!
Paradigmas de Programación (PLP) - Final 3/7/2026

Ejercicio 1 - Programación funcional

Se define el siguiente tipo algebraico para representar heaps binarios:

```
data Heap a = Vacio | Nodo Int a (Heap a) (Heap a)
  deriving (Show, Eq)
```

En cada nodo, el primer valor de tipo `Int` representa la **prioridad** del elemento almacenado, y el **segundo valor** representa el dato asociado. Diremos que un valor de tipo `Heap a` satisface el **invariante de min-heap** si, para cada nodo, su prioridad es menor o igual que la prioridad de cada uno de sus hijos no vacíos.

Por ejemplo, el siguiente heap satisface el invariante:

```
h1 :: Heap Char
h1 =
  Nodo 1 'a'
    (Nodo 3 'b' Vacio Vacio)
    (Nodo 5 'c'
      (Nodo 8 'd' Vacio Vacio)
      Vacio)
```

En cambio, el siguiente heap no lo satisface, porque el nodo con prioridad 4 tiene un hijo con prioridad 2:

```
h2 :: Heap Char
h2 =
  Nodo 4 'a'
    (Nodo 2 'b' Vacio Vacio)
    (Nodo 7 'c' Vacio Vacio)
```

No se exige ninguna propiedad de balanceo ni de completitud sobre la forma del árbol.

1.1 - Recursión estructural

Dar el tipo y definir una función `foldHeap`, que abstraiga el esquema de recursión estructural sobre heaps.

1.2 - Aplicación de funciones en los valores

Sin usar recursión explícita, dar el tipo y definir una función `mapHeap`, que aplique una función dada a todos los valores almacenados en el heap, sin modificar las prioridades ni la estructura.

1.3 - Recursión primitiva

Dar el tipo y definir una función `recHeap`, que abstraiga el esquema de recursión primitiva sobre heaps.

1.4 - Función prioridades

Sin usar recursión explícita, definir la función:

```
prioridades :: Heap a -> [Int]
```

que devuelva la lista de todas las prioridades almacenadas en el heap.

Por ejemplo:

```
prioridades h1
-- [1,3,5,8]
```

No importa el orden en que aparezcan las prioridades, siempre que estén todas y no se repitan artificialmente.

1.5 - Invariante del heap

Sin usar recursión explícita, definir la función:

```
esHeap :: Heap a -> Bool
```

que indique si el heap satisface el invariante de min-heap definido más arriba.

Se deberá usar alguno de los esquemas de recursión definidos en los incisos anteriores y justificar por qué resulta adecuado para esta función.

Ejercicio 2 - Cálculo lambda tipado

Se considera el cálculo lambda simplemente tipado visto en la materia, con valores booleanos, condicionales, funciones y las reglas usuales de tipado y evaluación *small-step*.

2.1 - Propiedades fundamentales

Explicar tres de las siguientes propiedades del cálculo lambda tipado, indicando en cada caso:

- qué afirma la propiedad;
- por qué es relevante para la evaluación de programas;
- qué tipo de error o comportamiento descarta.

Propiedades posibles:

1. Preservación de tipos.
2. Progreso.
3. Determinismo de la evaluación.
4. Unicidad de tipos.
5. Normalización en el cálculo lambda simplemente tipado.

No alcanza con nombrar las propiedades: se debe explicar su significado en términos de programas bien tipados y evaluación.

2.2 - Extensión de un cálculo

Se quiere extender el cálculo lambda tipado con una nueva construcción sintáctica. Explicar qué decisiones deben tomarse para que la extensión quede bien definida.

En particular, indicar qué debe definirse o revisarse en relación con:

1. la sintaxis de términos;
2. la sintaxis de tipos, si hiciera falta;
3. el conjunto de valores;
4. las reglas de tipado;
5. las reglas de evaluación;

Para cada punto, explicar brevemente qué podría salir mal si no se lo define de manera precisa.

2.3 - Una extensión no determinista

Se agrega al lenguaje una construcción $M \oplus N$, con la siguiente regla de tipado:

$$\frac{\Gamma \vdash M : \tau \quad \Gamma \vdash N : \tau}{\Gamma \vdash M \oplus N : \tau}$$

Además, se agregan las siguientes reglas de evaluación:

$$\frac{M \rightarrow M'}{M \oplus N \rightarrow M' \oplus N}$$

$$\frac{N \rightarrow N'}{V \oplus N \rightarrow V \oplus N'}$$

$$V_1 \oplus V_2 \rightarrow V_1$$

$$V_1 \oplus V_2 \rightarrow V_2$$

donde V , V_1 y V_2 representan valores.

Responder:

- a) Dar un ejemplo concreto de término cerrado y bien tipado cuya evaluación pueda terminar en dos valores distintos.
- b) Indicar cuál de las propiedades discutidas en el inciso 2.1 se pierde con esta extensión. Justificar.

- c) Indicar cuáles de las propiedades discutidas en el inciso 2.1 se conservan, al menos intuitivamente. Justificar.
- d) Explicar si la extensión introduce nuevos errores de tipado o si solamente cambia el comportamiento operacional del lenguaje.

2.4 - Recuperar determinismo

Proponer una modificación de las reglas de evaluación de $M \oplus N$ para que la evaluación vuelva a ser determinista.

La modificación debe conservar la regla de tipado anterior y debe seguir permitiendo evaluar términos de la forma $M \oplus N$.

Indicar claramente:

1. qué reglas se eliminan o reemplazan;
2. cuál sería el resultado de evaluar $true \oplus false$;
3. por qué con las nuevas reglas ya no hay dos próximos pasos posibles para un mismo término.

~~Indicar claramente:~~

2.5 - Preservación para la extensión

Justificar informalmente que la extensión con $M \oplus N$ satisface preservación de tipos.

Es decir, justificar que si:

$$\Gamma \vdash M \oplus N : \tau$$

y:

$$M \oplus N \rightarrow P$$

entonces:

$$\Gamma \vdash P : \tau$$

La justificación debe considerar los distintos tipos de reglas de evaluación de $M \oplus N$:

- reducción del lado izquierdo;
- reducción del lado derecho;
- selección del primer valor;
- selección del segundo valor.

2.6 - Progreso para la extensión

Analizar informalmente si la extensión con $M \oplus N$ satisface progreso.

En particular, explicar qué ocurre cuando:

1. M todavía puede reducir;
2. M ya es un valor y N todavía puede reducir;
3. ambos lados ya son valores.

Concluir si todo término cerrado y bien tipado de la forma $M \oplus N$ es un valor o puede dar un paso de evaluación.

Ejercicio 3 — Programación lógica y Resolución

Se quiere modelar una red académica de relaciones entre investigadores usando Prolog. Para ello, se cuenta con hechos de la forma:

`publicaron(A, B).`

que indican que A y B publicaron conjuntamente. Esta relación debe interpretarse como simétrica: si A publicó con B, entonces B publicó con A.

También se cuenta con hechos de la forma:

`dirigio(Director, Dirigido).`

que indican que Director dirigió o codirigió la tesis doctoral de Dirigido. Para calcular distancias académicas, la relación de dirección se considerará bidireccional y tendrá distancia 0.5. Es decir, aunque dos investigadores no hayan publicado juntos, si uno dirigió la tesis doctoral del otro, se considera que están a distancia académica 0.5.

Las publicaciones conjuntas tendrán distancia 1.

Se provee la siguiente base de conocimientos:

```
publicaron(erdos, renyi).
publicaron(erdos, turan).
publicaron(erdos, bollobas).
publicaron(erdos, graham).
publicaron(bollobas, riordan).
publicaron(graham, knuth).
publicaron(graham, patashnik).
publicaron(knuth, patashnik).
```

```
dirigio(fejer, erdos).
dirigio(fejer, turan).
dirigio(erdos, bollobas).
dirigio(lehmer, graham).
dirigio(church, turing).
dirigio(hall, knuth).
```

Además, se proveen los siguientes predicados auxiliares:

% coautor(?A, ?B)

% A y B son coautores si publicaron conjuntamente. La relación se considera simétrica.

```
coautor(A, B) :- publicaron(A, B).
```

```
coautor(A, B) :- publicaron(B, A).
```

% vinculo_academico(?A, ?B, ?D)

% A y B tienen un vínculo académico directo de distancia D. Publicar conjuntamente tiene distancia 1. La c

```
vinculo_academico(A, B, 1) :- coautor(A, B).
```

```
vinculo_academico(A, B, 0.5) :- dirigio(A, B).
```

```
vinculo_academico(A, B, 0.5) :- dirigio(B, A).
```

A partir de esta base, resolver los siguientes incisos.

3.1)

Definir el predicado:

descendiente_academico(+C, -D)

que, dado un científico C, instancie en D a cada uno de sus descendientes académicos, directos o indirectos.

Diremos que D es descendiente académico directo de C si vale:

```
dirigio(C, D).
```

Además, si C dirigió a X y X tiene como descendiente académico a D, entonces D también es descendiente académico de C.

El predicado debe poder devolver, por backtracking, hijos, nietos, bisnietos académicos, etc.

Por ejemplo:

```
?- descendiente_academico(fejer, D).
```

```
D = erdos ;
```

```
D = turan ;
```

```
D = bollobas ;
```

```
...
```


3.2)

Definir el predicado:

`camino_academico(+A, +B, -Camino, -D)`

que, dados dos científicos A y B, genere caminos académicos simples entre ellos.

Un camino académico puede usar tanto publicaciones conjuntas como vínculos de dirección doctoral. La lista **Camino** debe comenzar en A, terminar en B y no debe repetir científicos.

La distancia total D debe ser la suma de las distancias de los vínculos utilizados, considerando:

- distancia 1 para publicaciones conjuntas;
- distancia 0.5 para vínculos de dirección doctoral.

Por ejemplo, una consulta posible es:

?- `camino_academico(renyi, bollobas, C, D).`

Una respuesta posible es:

`C = [renyi, erdos, bollobas],`

`D = 1.5`

porque **renyi** publicó con **erdos**, y **erdos** dirigió a **bollobas**.

3.3)

Definir el predicado:

`distancia_academica(+A, +B, -D)`

que instancie D con la mínima distancia académica entre A y B.

Si hay varios caminos distintos con la misma distancia mínima, alcanza con devolver la distancia una sola vez.

Se debe considerar que:

`distancia_academica(A, A, 0).`

para cualquier científico A registrado en la base de conocimientos.

Ejemplos esperados:

?- `distancia_academica(erdos, renyi, D).`

`D = 1.`

?- `distancia_academica(erdos, bollobas, D).`

`D = 0.5.`

?- `distancia_academica(renyi, bollobas, D).`

`D = 1.5.`

3.4)

Analizar si el predicado `distancia_academica/3` definido en el inciso anterior es reversible en sus argumentos.

En particular, discutir qué ocurre con consultas como:

?- `distancia_academica(erdos, X, D).`

y:

?- `distancia_academica(X, Y, D).`

Justificar la respuesta distinguiendo entre:

- el significado declarativo del predicado;
- el comportamiento operacional esperable bajo la estrategia usual de Prolog.

3.5)

Considerar ahora la siguiente definición alternativa del predicado `relacionado/2`, que intenta indicar que dos científicos están conectados por algún camino académico:

```
relacionado(A, B) :-  
    relacionado(A, C),  
    vinculo_academico(C, B, _).
```

```
relacionado(A, B) :-  
    vinculo_academico(A, B, _).
```

Responder:

1. ¿La definición es declarativamente razonable?
2. ¿Qué ocurre en Prolog con la consulta siguiente?
`?- relacionado(erdos, renyi).`
3. Explicar por qué, bajo la estrategia usual de Prolog, la consulta puede no finalizar.
4. Mostrar cómo puede demostrarse `relacionado(erdos, renyi)` usando Resolución General.
5. Indicar cuál es la diferencia entre esta demostración y la ejecución de Prolog.